

**LAPORAN PRAKTIKUM
TUGAS 3
IMPLEMENTASI RESTFUL DAN GRAPHQL API
STUDI KASUS APLIKASI WISATAKU**



Disusun oleh:
[NAMA MAHASISWA]
NIM: [NIM]

**PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TINGGI MANAJEMEN INFORMATIKA DAN KOMPUTER
STMIK LOMBOK
PRAYA
2026**

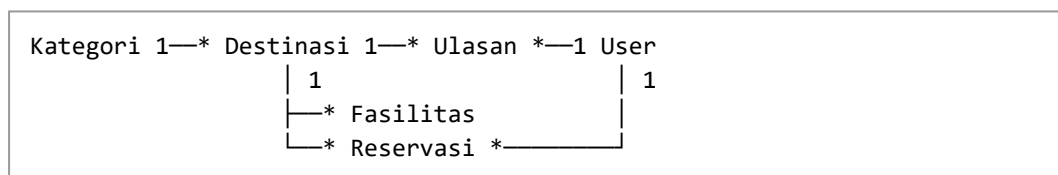
1. Ruang Lingkup

Tugas 3 mengimplementasikan REST API dan GraphQL API untuk resource Destinasi WisataKu (Bab 3 dan Bab 4 modul). Implementasi berada di aplikasi monolitik wisataku-api, dengan logika domain di `libs/domain` dan akses database di `libs/common`. Kode yang sama kemudian dipakai ulang oleh microservice pada Tugas 5.

Komponen	Lokasi
Skema dan migrasi database	<code>prisma/schema.prisma</code> , <code>prisma/migrations/</code>
Data contoh	<code>prisma/seed.ts</code>
PrismaService, PrismaModule (global)	<code>libs/common/src/prisma/</code>
Service domain, DTO, entity	<code>libs/domain/src/</code>
Controller REST dan resolver GraphQL	<code>wisataku-api/src/</code>
Skema GraphQL hasil generate	<code>wisataku-api/src/schema.gql</code>

2. Database: Prisma dan MariaDB

Prisma dikonfigurasi dengan provider `mysql` yang kompatibel dengan MariaDB. Koneksi dibaca dari environment `DATABASE_URL`.



Model	Field utama	Catatan
Kategori	<code>namaKategori</code> (unik)	Tabel terpisah sesuai tabel entitas modul
Destinasi	<code>nama</code> , <code>kategoriId</code> , <code>lokasi</code> , <code>deskripsi</code> , <code>hargaTiket</code> , <code>hargaAnak</code> , <code>ratingRata</code> , <code>createdAt</code>	<code>hargaAnak</code> ditambahkan pada Bab 7
Ulasan	<code>destinasiId</code> , <code>userId</code> , <code>rating</code> , <code>komentar</code> , <code>tanggal</code>	Index (<code>destinasiId</code> , <code>tanggal</code>) untuk mengambil ulasan terbaru
Fasilitas	<code>destinasiId</code> , <code>namaFasilitas</code>	

Contoh Bab 3 menyimpan kategori sebagai teks di tabel Destinasi. Proyek ini memakai model kategori terpisah sesuai tabel entitas modul (hal. 4), tetapi API tetap menerima dan mengembalikan kategori sebagai teks, misalnya "Pantai". `DestinasiService` memakai `connectOrCreate`, sehingga kategori baru dibuat otomatis saat destinasi ditambahkan.

Migrasi dibuat bertahap mengikuti modul (`init`, `add_user`, `add_reservasi`, `add_harga_anak`). `npm run db:seed` mengisi 5 kategori, 5 destinasi di Lombok, fasilitas, dan ulasan contoh. Nama destinasi nyata, sedangkan harga dan ulasan adalah data fiktif.

`PrismaService` meng-extend `PrismaClient` dan membuka/menutup koneksi pada `onModuleInit/onModuleDestroy`. `PrismaModule` diberi `@Global()`, sehingga service lain cukup meminta `PrismaService` di constructor.

3. REST API

3.1 Endpoint CRUD Destinasi

Method	Endpoint	Implementasi	Akses
GET	<code>/destinasi?kategori=&page=&limit=</code>	<code>findMany</code> dengan filter relasi kategori,urut terbaru	Publik
GET	<code>/destinasi/:id</code>	<code>findUnique</code> ; tidak ada → <code>NotFoundException</code> (404)	Publik
POST	<code>/destinasi</code>	<code>create + connectOrCreate</code> kategori	Admin
PATCH	<code>/destinasi/:id</code>	cek keberadaan, lalu update sebagian field	Admin
DELETE	<code>/destinasi/:id</code>	cek keberadaan, lalu delete; ada reservasi → 409	Admin
GET	<code>/destinasi/:id/ulasan</code>	ulasan terbaru lebih dulu	Publik
GET	<code>/destinasi/:id/fasilitas</code>	daftar fasilitas	Publik

Otorisasi admin dibahas di Tugas 4. Untuk Tugas 3 yang penting adalah perilaku CRUD dan penanganan error:

- `parseIntPipe` pada `:id`: GET `/destinasi/abc` → 400 tanpa query ke database.
- `NotFoundException` untuk id yang tidak ada, juga pada PATCH dan DELETE, sehingga PATCH `/destinasi/999999` → 404, bukan 500.

- ValidationPipe global menolak body yang tidak memenuhi DTO → 400.
- Menghapus destinasi yang sudah memiliki reservasi ditolak dengan 409. Relasi Reservasi → Destinasi memakai onDelete: Restrict agar riwayat pemesanan tidak ikut terhapus; error Prisma P2003 diubah menjadi ConflictException.

3.2 Contoh respons

Respons asli GET /v2/destinasi/1 dari gateway (30 September 2026). Endpoint v1 dan monolit mengembalikan field yang sama, dengan hargaTiket di tempat hargaDewasa/hargaAnak:

```
{
  "id": 1,
  "nama": "Pantai Kuta Mandalika",
  "kategori": "Pantai",
  "lokasi": "Kuta, Lombok Tengah",
  "deskripsi": "Pantai berpasir putih di kawasan ekonomi khusus Mandalika.",
  "ratingRata": 4.5,
  "createdAt": "2026-09-29T17:32:58.800Z",
  "hargaDewasa": 15000,
  "hargaAnak": 10000
}
```

Respons error 404:

```
{
  "message": "Destinasi dengan id 999999 tidak ditemukan",
  "error": "Not Found",
  "statusCode": 404
}
```

4. GraphQL API

4.1 Konfigurasi

GraphQL memakai pendekatan code-first (@nestjs/graphql + @nestjs/apollo, Apollo Server 5). Skema dibentuk dari kelas TypeScript bertanda @ObjectType() dan disimpan otomatis ke schema.gql. Endpoint POST /graphql; membuka GET /graphql di browser menampilkan Apollo Sandbox.

Kelas Destinasi, Ulasan, dan Fasilitas dipakai sekaligus sebagai skema GraphQL (@Field) dan skema Swagger (@ApiProperty), sehingga REST dan GraphQL tidak memiliki dua definisi data yang bisa berbeda.

4.2 Skema

```
type Destinasi {
  id: Int!
  nama: String!
  kategori: String!
  lokasi: String
  deskripsi: String
  hargaTiket: Float!
  ratingRata: Float!
  ulasan: [Ulasan!]!
  fasilitas: [Fasilitas!]!
}

type Query {
  destinasi(id: Int!): Destinasi!
  cariDestinasi(kategori: String): [Destinasi!]!
}

type Mutation {
  tambahUlasan(input: CreateUlasanInput!): Ulasan!
}

input CreateUlasanInput {
  destinasiId: Int!
  rating: Int!      # 1 sampai 5
  komentar: String!
}
```

4.3 Query, nested field, dan mutation

Query `destinasi(id).` `DestinasiResolver.findOne` memanggil `DestinasiService.findOne`, service yang sama dengan REST.

Nested field. `ulasan` dan `fasilitas` diisi dengan `@ResolveField`. Resolver hanya dijalankan bila client meminta field tersebut, sehingga query yang hanya meminta nama tidak menyentuh tabel `Ulasan` dan `Fasilitas`. `ulasan` dibatasi 3 ulasan terbaru, sesuai kebutuhan halaman detail.

```

query {
  destinasi(id: 1) {
    nama
    kategori
    ratingRata
    ulasan { rating komentar tanggal }
    fasilitas { namaFasilitas }
  }
}

```

Query cariDestinasi(kategori). Memanggil `DestinasiService.findAll({ kategori })`. Argumen opsional; tanpa argumen seluruh destinasi dikembalikan.

Mutation tambahUlasan(input). Hanya untuk role wisatawan (`GqlAuthGuard + RolesGuard`). `UlasanService.create` menjalankan satu transaksi Prisma: memastikan destinasi ada, menyimpan ulasan, menghitung rata-rata rating, lalu memperbarui `ratingRata` destinasi. Dengan transaksi, `ratingRata` tidak pernah berbeda dari data ulasan yang tersimpan.

```

mutation {
  tambahUlasan(input: { destinasiId: 1, rating: 5, komentar:
"Pemandangan luar biasa!" }) {
    id
    rating
    komentar
  }
}

```

Error GraphQL mengikuti status HTTP aslinya: destinasi tidak ada → `extensions.code = "NOT_FOUND"`, rating di luar 1–5 → `BAD_REQUEST`, tanpa token → `UNAUTHENTICATED`. Pemetaan `NOT_FOUND/CONFLICT` ditambahkan di `libs/common/src/graphql/graphql-options.ts` karena NestJS secara bawaan hanya memetakan 400/401/403.

5. Perbandingan Jumlah Request REST vs GraphQL

Pertanyaan latihan Bab 4.4 langkah 6: berapa request HTTP yang dibutuhkan untuk menampilkan "detail destinasi lengkap" (data destinasi, ulasan, dan fasilitas)?

Pendekatan	Request	Keterangan
REST murni (Bab 3)	3	GET /destinasi/1, GET /destinasi/1/ulasan, GET /destinasi/1/fasilitas
GraphQL (Bab 4)	1	POST /graphql dengan query <code>destinasi(id: 1) { ... ulasan { ... } fasilitas { ... } }</code>
REST agregasi di gateway (Bab 6)	1	GET /destinasi/1/lengkap, gateway memanggil tiga pesan TCP secara paralel

Dengan REST murni, client melakukan tiga round-trip dan menerima semua field meskipun hanya sebagian yang ditampilkan. Pada koneksi mobile dengan latensi 200 ms per round-trip, tiga request berurutan sudah memakan sekitar 600 ms sebelum waktu proses server dihitung (angka ini ilustrasi, bukan hasil pengukuran). GraphQL menyelesaikannya dalam satu request dan hanya mengirim field yang diminta. Kekurangannya, query GraphQL lebih sulit di-cache di level HTTP karena semuanya memakai POST /graphql.

Di sisi server jumlah query database tetap tiga: satu untuk destinasi, satu untuk ulasan, satu untuk fasilitas. Yang berkurang adalah jumlah round-trip client-server.

6. Bukti Pengujian API

Pengujian dijalankan otomatis, bukan hanya manual:

Bukti	File	Hasil
E2E REST & GraphQL monolit	test/destinasi.e2e-spec.ts, output di docs/tugas-4/test-results/e2e-test.txt	Lulus pada run pengujian 30 September 2026 (bagian dari 41/41)
Smoke test HTTP ke server berjalan	Dicatat di docs/recovery-status.md	Monolit 32/32, gateway 37/37
Unit test service domain	libs/domain/src/**/*.spec.ts	Lulus (bagian dari 32/32)

Skenario GraphQL yang diuji e2e: tambahUlasan tanpa token → UNAUTHENTICATED; dua ulasan (rating 5 dan 4) oleh wisatawan lalu destinasi(id) mengembalikan ratingRata 4.5, 2 ulasan, dan daftar fasilitas (kosong untuk destinasi uji); rating 9 → BAD_REQUEST; destinasi(id: 999999) → NOT_FOUND; cariDestinasi(kategori: "Pantai") hanya berisi kategori Pantai.

6.1 Bukti Uji REST dengan Postman

Screenshot belum tersedia. Bagian ini sengaja tidak diisi gambar tiruan.

ACTION REQUIRED — SCREENSHOT POSTMAN

Jalankan `npm run start:api`, lalu ambil screenshot Postman untuk request berikut dan simpan di `docs/tugas-3/screenshots/`:

1. GET `http://localhost:3000/destinasi?kategori=Pantai` → 200
2. POST `http://localhost:3000/auth/login` (akun admin) → 200 dengan `access_token`
3. POST `http://localhost:3000/destinasi` dengan header `Authorization: Bearer <token admin>` → 201
4. GET `http://localhost:3000/destinasi/999999` → 404

6.2 Bukti Uji GraphQL dengan Apollo Sandbox

Screenshot belum tersedia. Bagian ini sengaja tidak diisi gambar tiruan.

ACTION REQUIRED — SCREENSHOT APOLLO SANDBOX

Buka `http://localhost:3000/graphql` di browser, lalu ambil screenshot berikut dan simpan di `docs/tugas-3/screenshots/`:

1. Query `destinasi(id: 1)` dengan field `nama`, `ratingRata`, `ulasan`, `fasilitas`
2. Query `cariDestinasi(kategori: "Pantai")`
3. Mutation `tambahUlasan` dengan header `Authorization: Bearer <token wisatawan>`

7. Kesimpulan

1. CRUD Destinasi berjalan terhadap MariaDB melalui Prisma, dengan validasi input, 400 untuk id bukan angka, 404 untuk data yang tidak ada, dan 409 untuk penghapusan yang melanggar riwayat reservasi.

2. GraphQL menyediakan `destinasi(id)`, `cariDestinasi(kategori)`, dan `tambahUlasan(input)`, dengan ulasan dan fasilitas sebagai nested field yang dieksekusi hanya bila diminta.
3. REST dan GraphQL memakai service domain dan kelas entity yang sama, sehingga perilaku dan bentuk data keduanya konsisten.
4. Untuk detail destinasi lengkap, REST murni membutuhkan 3 request sedangkan GraphQL 1 request.